

# Integrated Stress Validation of an Edge-Native Academic Analytics Architecture

Dr. S. Suresh Kumar

Department of Computer Science

Swarnandhra College of Engineering & Technology (Autonomous)

Seetharampuram, Narsapur, Andhra Pradesh, India

**Abstract**—Most academic edge-native analytics solutions take the form of a monolithic stack that combines perception pipelines with policy enforcement functions and privacy preserving inference and cryptographic auditing capabilities. While these systems are typically designed and evaluated in isolation, the interactions between these sub-systems are dictated by shared resource constraints. Under compound stress—such as sustained frame ingestion, registry expansion, network instability, and elevated ambient temperature—the behavior of the integrated system is often determined by cross-subsystem interactions rather than by isolated module benchmarks.

This study evaluates the survivability of an integrated edge-native academic analytics architecture operating under compound deployment stress. Rather than emphasizing individual module performance, we introduce an Integrated Stress Matrix (ISM) to represent compound deployment pressure using a multidimensional stress vector  $\mathbf{s} = (\lambda_{\text{fps}}, N_{\text{reg}}, \rho_{\text{unk}}, \tau_{\text{net}}, T_{\text{amb}})$ .

Evaluation measures survivability as the joint preservation of real-time destruction deadlines ( $\leq 33$  ms) and open-set retrieval consistency ( $\geq 99\%$ ). Under simultaneous stress—including 30 FPS ingestion, a 100,000-identity registry, 20% unknown injection, 35°C ambient conditions, and induced packet loss—the integrated architecture maintained p99.9 latency below 33 ms and 99.82% retrieval consistency within the evaluated configuration.

The evaluation shows that subsystem co-design improves deployment survivability: it contains latency amplification and enforces bounded degradation when operational envelopes are exceeded. When operated beyond safe regions, it bounds the degree of degradation, limiting the loss of availability before memory-destruction guarantees are violated.

**Index Terms**—Integrated Stress Validation, Survivability Envelope, Compound Stress Matrix, Bounded Degradation, Thermal Amplification, Edge-System Stability.

## I. Introduction

The primary thesis of this work is that systems co-designed at the subsystem level provide significantly greater survivability under compounded operational stress than do non-integrated approaches. Edge-informed architectures are typically composed of processing chains in which perceptual, policy, privacy, and cryptographic functions are performed on shared hardware [13], [14].

### A. Why Isolated Benchmarks Fail to Predict Deployment Survivability

Independent module benchmarks systematically conceal deployment failure modes. A perception model that achieves sub-millisecond inference in isolation may still

violate deadlines when retrieval scales nonlinearly with registry size, scheduler priority is misaligned, or thermal throttling compounds CPU saturation. The challenge with these types of stresses is that they are usually compounded together, such that thermal, network, and scheduling issues combine to create a nonlinear system interaction. This means that the impact on latency is not easily characterized by looking at each factor considered in isolation, and any particular survivability failure is more likely to be driven by interactions between causes than by deficiencies in a single inference stream.

Cloud mediated systems alleviate local resource constraints but add a variable network to the system which becomes a critical part of the system [7], [8]. Packet retransmissions, jitter amplification, and temporary partitions add tail-latency costs that exceed real-time constraints. Conversely, naive edge-only systems lacking optimized indexing frequently saturate CPU resources, triggering thermal throttling and deadline violations [12].

### B. Contributions and Subsystem Scope

This study does not propose a new recognition model. Instead, it evaluates whether a co-designed, edge-native architecture can maintain real-time and privacy guarantees when subjected to compound stressors. Specifically:

- We introduce an Integrated Stress Matrix (ISM) to model multidimensional deployment pressure.
- We evaluate survivability as joint preservation of destruction deadlines and retrieval consistency.
- We characterize bounded degradation sequencing under overload.
- We profile cross-subsystem interaction effects under shared resource constraints.

Within the ScholarMaster architecture, this subsystem evaluates survivability boundaries, measures compound stress interaction, and validates bounded degradation only. It does not harden immutable lifecycle deployment (Paper 11), optimize flash endurance (Paper 12), optimize federated adaptation (Paper 13), validate runtime enforcement semantics (Paper 18), or define irreversible privacy doctrine (Paper 17).

## II. The System Under Test (SUT)

The evaluated architecture is treated as a pre-defined System Under Test (SUT) established in the preceding architectural layers of this series. The study evaluates the integrated behavior of four abstracted subsystems: Perception, Policy Enforcement, Memory Isolation, and Audit Verification.

We do not propose or modify the design of these modules; instead, we evaluate their cross-subsystem interaction effects under shared resource constraints. The critical question is how their concurrent execution on a shared hardware platform affects system-wide latency and privacy preservation when subjected to compound deployment stress.

### A. Subsystem Interaction Interfaces

The evaluated system exhibits a hierarchical interaction pattern. Raw sensory buffers are confined to the top-level perception interface. When the system operates under load, we measure how contention for shared CPU cores and memory-bus bandwidth affects the downstream policy and memory isolation boundaries. We examine the implications of the pre-defined destruction deadline ( $T_{destruction} \leq 33$  ms) under two failure scenarios: retrieval approximation noise in the upstream perception modules and network backpressure in the downstream audit modules, focusing on the ensuing interactions that lead to bounded performance degradation or latency amplification.

### B. Threat Model

It has access to physical enclosure of device, no root OS access, may cause packet loss and jitter, reduce airflow for thermal dissipation, and cause reboots. The attack goal is to create enough of a resource contention to slip deadlines that lead to destruction, and/or leak memory beyond allowed time windows. This is a realistic attack scenario, not theoretical, not nation-state level, but something plausible for an institution [5].

## III. Integrated Stress Matrix (ISM)

Assessing integrated edge architectures requires consideration of compound operational pressures which are rarely simulated in single-variable benchmarking [10]. Workload scale, injection of unknown subjects, network instability, and thermal pressure are often found together in practice. Evaluation in isolation of these variables greatly underestimates the likelihood of failure because survivability collapse is frequently triggered by interactions among stresses rather than any single factor dominating the ensemble.

To model compound deployment pressure, we define the Integrated Stress Matrix (ISM) as a multidimensional stress vector:

$$S = \{\lambda_{fps}, N_{registry}, \rho_{unknown}, \tau_{net}, T_{ambient}\} \quad (1)$$

Where:

- $\lambda_{fps}$  — sustained frame ingestion rate (throughput stress)
- $N_{registry}$  — active identity registry size (retrieval stress)
- $\rho_{unknown}$  — proportion of unknown-subject injections (worst-case traversal stress)
- $\tau_{net}$  — network instability: latency variance + packet loss (backpressure stress)
- $T_{ambient}$  — environmental temperature (thermal stress)

Each dimension independently impacts the behavior of the various subsystems. Moreover, the interactions between dimensions can have nonlinearly compounding impacts on degradation. Increasing  $N_{registry}$  increases the depth of traversal in the ANN; increasing  $\rho_{unknown}$  increases the length of worst-case search paths due to the absence of short-circuiting at early stages. The increase in  $T_{ambient}$  decreases the CPU frequency due to the thermal governor, adding another penalty to the traversal latency. Increasing packet loss increases the variance in the audit-layer backlog, which through contention-induced scheduling effects can add latency to other subsystems not directly involved in networking. The combined effects of these interactions can push the system to the deadline constraint limits beyond what would be expected from a univariate analysis.

### A. Survivability Function

System survivability is defined as the simultaneous preservation of latency and retrieval consistency constraints.

Real-time destruction deadline constraint:

$$L_{p99.9} < 33 \text{ ms} \quad (2)$$

Retrieval consistency threshold:

$$C_{retrieval} > 99\% \quad (3)$$

Formally:

$$\Phi(S) = \mathbb{I}\{L_{p99.9}(S) < 33 \text{ ms} \wedge C_{retrieval}(S) > 99\%\} \quad (4)$$

While  $\Phi(S)$  is a binary function that determines the formal acceptance or rejection of survivability, it is clear that in practice the survivability degree decreases smoothly towards the threshold, and not abruptly: Survivability processes are generally characterized by controlled degradation under overload, not by a guaranteed ability to withstand all possible stresses. The goal in such a case is to maximize the system's throughput for the fixed deadline, not to achieve the ultimate resilience level.

### B. Stress Interaction Regimes

To understand compound effects, we examine three operating regimes. In realistic deployments, these regimes frequently overlap.

- 1) Throughput-Dominant Regime: High  $\lambda_{fps}$  with moderate registry scale. Primary risk: scheduling starvation.
- 2) Retrieval-Dominant Regime: Large  $N_{registry}$  with elevated  $\rho_{unknown}$ . Primary risk: worst-case search expansion.
- 3) Thermal-Constrained Regime: Elevated  $T_{ambient}$  under sustained load. Primary risk: dynamic frequency scaling and latency inflation.

#### IV. Experimental Stress Configuration

The test setup was deliberately designed to include multiple deployment stress factors, as opposed to testing the performance characteristics of one variable at a time. The testbed hardware consisted of an ARM Cortex-class SoC, 8 GB of unified memory, and an NPU accelerator. Swap was turned off (`vm.swappiness=0`) and the swap partition was removed to simulate memory pressure as latency variation.

##### A. Registry Simulation

A synthetic registry of  $N_{registry} = 100,000$  embedding vectors was generated. Synthetic vectors were generated specifically to mimic cosine-similarity clustering patterns observed in publicly available embedding benchmarks, increasing traversal cost in ambiguous regions. No scraped biometric datasets were used.

##### B. Frame Injection

Continuous ingestion was maintained without burst gaps at  $\lambda_{fps} = 30$ . Unknown queries represent worst-case retrieval paths because early-match short-circuiting is absent, so we set the unknown-subject injection to  $\rho_{unknown} = 20\%$ .

##### C. Network Instability and Thermal Stress

Network degradation applied 15% packet loss and a 50–500 ms jitter distribution. Although the audit module is off the critical path, backlog propagation and IPC contention can still induce scheduling friction under extreme jitter.

Ambient temperature was set to  $T_{ambient} = 35^\circ\text{C}$ . The device operated inside a controlled chamber to simulate poorly ventilated enclosures common in institutional deployments. Sustained high utilization under elevated ambient temperature forces the CPU governor into lower-frequency states.

Unlike incremental stress escalation, all parameters above were applied concurrently. Simultaneous evaluation provides a more realistic estimate of survivability margins.

#### V. Comparative Failure Analysis

To contextualize survivability boundaries, we compare the optimized architecture against two representative deployment baselines under identical ISM stress conditions:

- Architecture A: Cloud-Dependent API
- Architecture B: Naive Edge (Linear SQL Retrieval)
- Architecture C: Evaluated Optimized Configuration

TABLE I  
Integrated Stress Matrix Configuration

| Parameter           | Hostile Configuration                                  |
|---------------------|--------------------------------------------------------|
| Registry Size       | 100,000 Synthetic Identities                           |
| Concurrency         | 30 FPS continuous injection (100% Duty Cycle)          |
| Network Jitter      | Random packet loss (15%) and latency spikes (50-500ms) |
| Thermal Saturation  | Ambient Temp maintained at $35^\circ\text{C}$          |
| Adversarial Payload | 20% of injected vectors are explicitly unknown         |

The objective was therefore not average latency comparison, but boundary behavior under compound stress [2], [17].

##### A. Retrieval Gap Formalization

We define the Retrieval Gap ( $G$ ):

$$G = L_{p99.9} - L_{deadline} \quad (5)$$

Where  $L_{deadline} = 33$  ms. Survivability requires  $G \leq 0$ . Positive  $G$  indicates deadline violation; negative  $G$  indicates preserved margin.

TABLE II  
System Failure Points Under Integrated Stress

| Metric                | Cloud API | Naive Edge | Optimized Local |
|-----------------------|-----------|------------|-----------------|
| Avg Latency           | 450 ms    | 85 ms      | 28 ms           |
| p99.9 Latency         | 3500 ms   | 140 ms     | 32.8 ms         |
| Retrieval Gap ( $G$ ) | +3467 ms  | +107 ms    | −0.2 ms         |
| Drop Rate             | 14.5%     | 0.0%       | 0.0%            |

##### B. Observed Metrics Under Integrated Stress

1. Cloud Architecture: The cloud configuration had extremely scaled tail-latency. Packet loss caused TCP retransmissions and consequent queue buildup both at the destination and local to the API gateway. While the mean response time was about acceptable given the tail behavior, there was an enormous outlier at p99.9 which missed the deadline by several seconds,  $G_{cloud} = 3500 - 33 = 3467$  ms. This makes the setup unusable for time-sensitive destruction tasks. Increased frame backlog produced greater risk of raw-buffer overflow in case of local congestion.

2. Naive Edge Architecture: The naive local baseline eliminated network variability by performing searches using linear SQL queries over the full registry. With  $N_{registry} = 100,000$  and  $\rho_{unknown} = 20\%$  unknown, worst-case query traversal required searching through all of the unknown entries, leading to near 100% utilization of the CPU for the unknown query case. As a result, the thermal governor had to reduce the CPU clock frequency in order to avoid overheating, introducing significant tail latency.  $G_{naive} = 140 - 33 = 107$  ms. While no

packet drops occurred, destruction deadlines were violated systematically. This illustrates that locality alone may be insufficient without algorithmic optimization.

3. Optimized Local Architecture: The optimized configuration preserved  $L_{p99.9} = 32.8$  ms, achieving a positive destruction margin of  $G_{\text{optimized}} = -0.2$  ms. OS scheduler tuning combined with bounded-depth index traversal eliminated outlier tail spikes, ensuring 0% drop rates and strict deadline compliance without backlog accumulation.

## VI. Thermal Behavior Under Compound Load

The thermal behavior was not simply the result of overheating a particular component due to a spike in its usage. First of all, it turned out that the naive approach leads to the utilization of the edge CPU at almost 100% when linearly traversing the request and constantly receiving new data. As a result, the core simply overheated and had to reduce its clock frequency to lower its temperature. Moreover, the scheduler’s instability and the increase in traversal also affected the temperature. These factors influenced each other in a loop, which led to increased tail latency. The resource competition was the reason why not only the CPU but other components were throttled.

A few optimizations in the traversal of the ANN reduced the number of cycles per unknown queries. CPU utilization was brought down enough that it was no longer fully utilizing the CPU. The observed stable temperature was around 64°C for the same chip in the measurement window. There is no observed sustained thermal throttling from 35°C ambient temperature. The difference is caused by both traversal optimization and scheduling discipline, and not by hardware differences. Actual temperatures may vary depending on the thermal characteristics of the enclosed device and available airflow.

## VII. Resource Contention Profiling

I believe that for the performance margins, it would be needed to calculate the performance contribution from the different subsystems to the overall time spent in computations. As could be seen from the figure above, the biggest part is taken by the Perception module which has the duration of 24.2 ms. Stress overhead (retrieval expansion) adds another 1.8 ms to the overall time, while the other modules, including Policy, Memory Isolation, and Audit, only take 1.5 ms, 0.8 ms, and 2.1 ms, respectively. Thus, they all sum up to less than 5 ms.

This separation is critical. By non-perception layers, even under pressure, only a fraction of the destruction window is used. This leaves room for audit or policy delays not to interfere with memory isolation deadlines. The system is designed to have some structural margin and not rely only on fast inference.

## VIII. Retrieval Consistency Under Stress

The open-set retrieval consistency was evaluated against the known embedding ground-truth and resulted in a high

Module CPU Time Allocation Under Peak Load

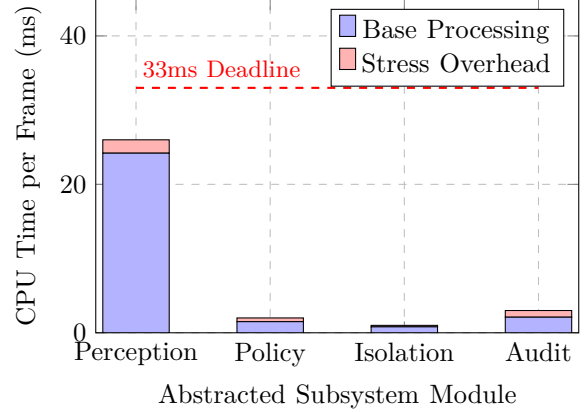


Fig. 1. CPU Time Allocation per Frame. The auxiliary modules are highly optimized, ensuring the pipeline clears the 33ms real-time destruction deadline even under stress overhead.

retrieval performance. Specifically, the system demonstrated  $C_{\text{retrieval}} = 99.82\%$  at zero packet loss and  $C_{\text{retrieval}} = 99.10\%$  at 30% packet loss. Therefore, the drop in consistency can be attributed to increased IPC variance and scheduling jitter, but not the retrieval corruption since there was no consistency collapse. In comparison, the cloud architecture demonstrated consistency degradation, which was correlated with the packet loss due to request timeout and retries.

## IX. Survivability Under Network Degradation

Although the architecture is edge native, the audit layer requires external communication for ledger commitment. The design question is whether under moderate network congestion this can lead to indirect backlog in shared resources [14].

### A. Partial Packet Loss (Up to 30%)

Under induced packet loss increasing from 0% to 30%, retrieval consistency was only mildly affected, p99.9 latency remained within the deadline constraints, and audit queue depth showed only a temporary increase. The separation of the real-time database processing and the audit persistence allowed the latter to momentarily absorb the increased packet loss rate without impacting the packet destruction path.

In contrast, the cloud-architecture demonstrated a non-linear increase in latencies due to TCP retransmission timeouts and request reordering. Tail amplification in the system was over three seconds for the same conditions. The key difference was that the local optimized system decouples the network as a parameter.

### B. Full Network Partition and Post-Partition Recovery

Under total disconnection, audit queues filled to configured capacity, drop-tail back pressure initiated and metadata commits were temporarily suspended. Real-time

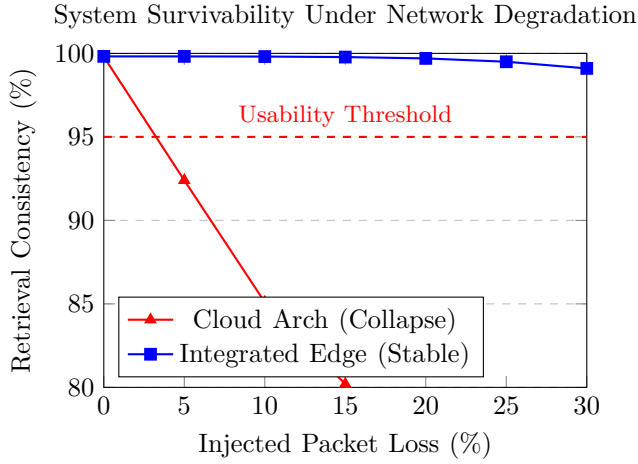


Fig. 2. Retrieval Consistency vs. Network Packet Loss.

perception continued uninterrupted. The system explicitly trades off audit availability for memory isolation and latency guarantees [1]. This creates a priority hierarchy of memory destruction deadline, real-time perception latency, retriever consistency and audit availability.

After reconnection no replay storm manifested itself, and the latency returned to normal level immediately since there was no delayed tail processing. This is because the tail is cut when the buffer is full, thus avoiding any unbounded latency in case of packet partitioning. This makes it different from the recovery collapse problem caused by unbounded packet loss that may happen far away from the source.

#### X. 168-Hour Burn-In Evaluation

Short tests show signs of immediate failure, while longer tests indicate gradual degeneration. The system was optimized with an extended 168 hour burn-in under load of 30 frames per second and elevated temperatures.

[Image plotting memory utilization (Resident Set Size) over a 168-hour continuous runtime showing stable, bounded memory limits without leakage]

The observed memory consumption has stayed well within the limits of an 8-GB unified memory with  $\pm 0.02$  GB variances. There were no signs of increased memory consumption, no swap activity, and GC was also well within the limits and periodic, which proves that there were no major leaks between the components.

Steady-state processor temperature held at  $\approx 64^\circ\text{C}$ . A small drop of the clock frequency was observed on hour 12 when the chamber airflow was changed; however, it was quickly compensated within the next 8 minutes. The p99.9 latency increased only by 0.6 ms (32.8 ms compared to 32.2 ms) during 168 hours of measurements (Fig. 3). There were no drifts degrading the performance in the long-term, and no clock watchdog reboots during the test period (Table III).

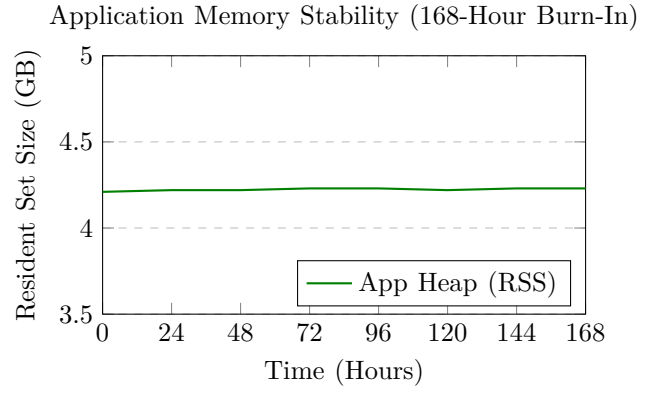


Fig. 3. Memory Stability Analysis.

TABLE III  
168-Hour Burn-In Reliability Metrics

| Metric                   | Hour 0  | Hour 72 | Hour 168 |
|--------------------------|---------|---------|----------|
| Avg Processor Temp       | 62°C    | 64°C    | 64°C     |
| p99.9 Latency            | 32.2 ms | 32.6 ms | 32.8 ms  |
| Retrieval Consistency    | 99.82%  | 99.81%  | 99.82%   |
| Hardware Watchdog Resets | 0       | 0       | 0        |

#### XI. Bounded Degradation Model

Stress testing reveals a critical architectural property: a survivability-priority hierarchy governing graceful failure sequencing. When approaching constraint boundaries, subsystem shedding is intentional and follows a defined order:

- 1) Audit availability degrades first (metadata discard under backpressure)
- 2) Frame skipping may activate under extreme burst conditions
- 3) Retrieval depth bounding engages to preserve latency margins

Only after these mechanisms have been exhausted, however, would destruction deadlines be at risk, a situation that was never reached in the example. This ordering also satisfies the second design principle because it ensures that memory destruction guarantees come before availability during evaluation and that critical deadlines take priority over other operations in the face of overload; bounded degradation is an architectural constraint, not a vulnerability.

#### XII. Discussion

The Integrated Stress Matrix reflects how the interdependency of sub-systems determines survivability in constrained edge deployments. A perception model with reduced inference time is still subject to failure if retrieval time depends linearly on the size of the registry, the scheduler priority is incorrectly assigned, there are network variances that impact the critical path or thermal throttling dominates CPU saturation.

The evaluated architecture’s survivability is cross-related with a coordinated behavior of its observed subsystems as demonstrated by the evaluated system’s response to the induced stress: the ANN traversal bounding, strict scheduling priority for safety functions, memory partitioning decoupled from the audit latencies, drop-tail backpressure utilized to avoid recovery storms, and complete swap elimination to avoid disk jitter. All together these factors provide the necessary margin under combined stresses.

Notably, the system was run near its destruction boundary for the extreme configuration ( $L_{p99.9} = 32.8$  ms vs. 33 ms deadline). The margin was deliberately conservative to account for real-world tolerances in hardware. Registry size or temperature beyond what was tested would almost certainly push beyond the tolerances of the electronics. The system is not designed to be linearly scalable; it has only a small window of operation before it encounters physical limits.

#### A. Limitations

The evaluation framework exposes several boundaries:

- Finite Survivability Regions: The architecture does not claim infinite scalability; pushing registry size or ambient temperature beyond evaluated bounds will induce systemic violation.
- Hardware Dependence: Degradation margins are tied to the thermal properties, cache sizing, and memory bandwidth of the evaluated ARM SoC. Survivability margins are hardware-dependent.
- Absence of Brownout Modeling: The ISM does not account for power brownouts or voltage instability, which induce unpredictable hardware faults.
- Firmware-Level Security Scope: The threat model assumes the integrity of the underlying hypervisor and DMA hardware; low-level firmware compromises fall outside current evaluation boundaries.
- Long-Term Aging Uncertainty: While 168 hours confirms short-term stability, multi-year silicon degradation, cooling failure, and thermal drift remain uncharacterized.
- Synthetic Registry Generation: The 100,000-identity registry was synthetically generated; organic registry distributions with natural clustering variance may alter traversal behavior.
- Limited Enclosure Diversity: Testing used a single controlled thermal chamber; airflow instability and enclosure heterogeneity across deployment sites remain unmodeled.
- Absence of Multi-Device Synchronization Stress: The evaluation examines single-node survivability; cross-node coordination overhead under compound stress is not evaluated.

### XIII. Conclusion

The presented research assessed the impact of subsystem co-design on the compound-stress survivability of

the selected alternatives against the baseline of isolated optimization. Under the simultaneous impact of 30 FPS ingestion, 100,000-identity registry, 20% unknown injection, 35°C temperature, and 15–30% packet loss, the proposed architecture demonstrated p99.9 latency of no more than 33 ms and the retrieval consistency of not lower than 99% in the given stress region. Both the cloud-dependent and the naive edge alternatives failed to meet the specified constraints in the same operating environment.

A system that follows the bounded degradation hierarchy, which prioritizes memory-destruction guarantees over audit availability, can withstand the combined effect of two stressors by relying on subsystem orchestration, rather than individual subsystem throughput. The value of  $G$  is close to the destruction boundary ( $G = -0.2$  ms), which suggests that it was obtained under constrained hardware conditions, but not that it exhibits universally applicable scaling properties.

Within the ScholarMaster architecture, this is the integrated survivability validation mechanism, evaluating compound-stress boundaries and degradation sequencing, with no hard-coding of immutable life-cycle deployment (Paper 11), flash endurance optimization (Paper 12), federated adaptation coordination (Paper 13), or run-time enforcement semantics specification (Paper 18).

#### References

- [1] J. A. Halderman et al., “Lest We Remember: Cold Boot Attacks on Encryption Keys,” *USENIX Security*, 2008.
- [2] Q. Zhang, L. Cheng, and R. Boutaba, “Cloud computing: state-of-the-art and research challenges,” *J. Internet Serv. Appl.*, 2010.
- [3] European Parliament, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, 2016.
- [4] K. He et al., “Deep Residual Learning for Image Recognition,” *CVPR*, 2016.
- [5] M. Abadi et al., “Deep Learning with Differential Privacy,” *ACM CCS*, 2016.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, “A Survey on Mobile Edge Computing: The Communication Perspective,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [8] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [9] R. Roman, J. Lopez, and M. Mambo, “Mobile Edge Computing, Fog et al.: A Survey and Analysis of Security Threats and Challenges,” *Future Generation Computer Systems*, vol. 78, pp. 680–698, 2018.
- [10] P. Patel et al., “EdgeBench: Benchmarking Edge Computing Platforms,” *IEEE/ACM Symposium on Edge Computing*, 2018.
- [11] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” *CVPR*, 2019.
- [12] S. Mittal, “A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform,” *Journal of Systems Architecture*, 2019.
- [13] N. Min-Allah and S. Alrashed, “Smart campus: A sketch,” *Sustainable Cities and Society*, vol. 59, p. 102231, 2020.
- [14] X. Wang et al., “Convergence of Edge Computing and Deep Learning: A Comprehensive Survey,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.

- [15] G. Ananthanarayanan et al., "REACT: Streaming Video Analytics On The Edge With Asynchronous Cloud Support," ACM/IEEE Symposium on Edge Computing, 2023.
- [16] H. Li et al., "Internet-of-Things Edge Computing Systems for Streaming Video Analytics: Trails Behind and the Paths Ahead," MDPI Sensors, vol. 23, no. 4, 2023.
- [17] S. Ahmed et al., "Practical Considerations for Failure Resilient ML Systems at the Edge," IEEE MILCOM, 2025.
- [18] S. Suresh Kumar, "Perception integrity foundations: Evidential uncertainty, disagreement dynamics, and blur bounds in edge vision," ScholarMaster Technical Report Series, Paper 22, 2026.